

The Evolution of AI Agents and Modern AI Engineering

Author: Mubshra Rehman **University:** University of Central Punjab, Faisalabad **Program:** AI Agent Fellowship 2026

1. Introduction

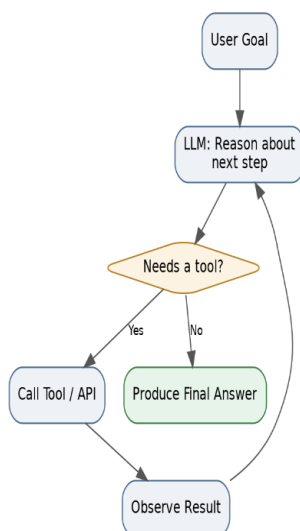
The rise of large language models (LLMs) has shifted software engineering from writing explicit logic to designing systems that reason, plan, and act. This report traces that evolution from LLMs as simple text generators, to autonomous agents capable of using tools, remembering context, planning multi-step tasks, and collaborating with other agents.

2. LLM Applications

Early LLM applications were single-turn: a prompt goes in, text comes out. This pattern powers chatbots, summarizers, code assistants, and content generators. The limitation is statelessness each call is isolated, with no memory of past interactions and no ability to affect the outside world.

3. AI Agents

An AI agent is an LLM wrapped in a control loop that lets it **perceive, decide, and act** repeatedly, rather than answering once and stopping. Instead of “answer this question,” the goal becomes “achieve this objective,” and the agent decides for itself which steps to take.



4. Tool Calling

Tool calling (also called “function calling”) lets an agent invoke external functions a search engine, a database query, a calculator, a code interpreter rather than answering purely from memory. The LLM outputs a structured request (e.g., a JSON object naming a function and its arguments); the surrounding application executes that function and feeds the result back into the model’s context.

5. Memory

Because a single LLM call has a limited context window, agents need a memory system to retain information across turns, sessions, or tasks. Memory is generally split into:

- **Short-term (working) memory:** the current conversation or task context, held directly in the prompt.
- **Long-term memory:** information persisted outside the model — often in a vector database — and retrieved when relevant (a form of RAG applied to the agent’s own history).
- **Episodic memory:** records of past actions and their outcomes, used to avoid repeating mistakes.

A practical example is an onboarding agent that stores a client’s prior form submissions in a vector store and retrieves relevant context in later conversations, rather than asking the client to repeat themselves.

6. Planning

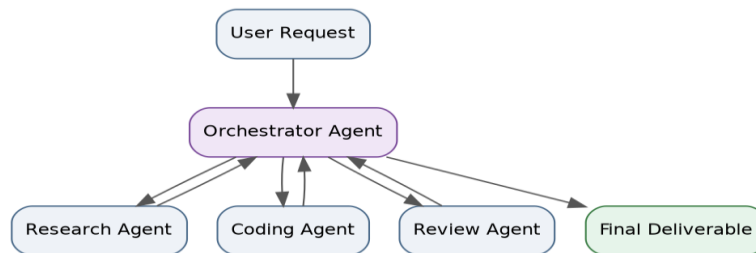
Planning is what lets an agent break a large, ambiguous goal into an ordered sequence of smaller, executable steps.

Techniques that emerged to improve planning reliability include:

- **Chain-of-Thought prompting** — asking the model to reason step by step before acting.

7. Multi-Agent Systems

As tasks grow more complex, a single agent handling everything becomes inefficient and error-prone. Multi-agent systems split work across specialized agents that collaborate for example, a “planner” agent, a “researcher” agent, and a “writer” agent, each with a narrow role and its own tools.



8. Future of AI Engineering

Several trends are shaping where this field goes next:

- **Standardization of tool interfaces** (MCP and similar protocols) will make agents less dependent on custom integration code, the same way REST/HTTP standardized web APIs.
- **Security and governance** are catching up to capability: rate limiting, input sanitization, and audit logging are shifting from afterthoughts to first-class requirements as agents gain the ability to take real-world actions.
- **Convergence toward fewer, more mature frameworks** — the ecosystem is consolidating (e.g., Microsoft merging AutoGen into a unified Agent Framework) rather than continuing to fragment.

References

1. Yao, S., et al. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629.
2. Anthropic (2024). Introducing the Model Context Protocol. anthropic.com/news/model-context-protocol.
3. LangChain AI. LangGraph Documentation. github.com/langchain-ai/langgraph.
4. FoundationAgents. MetaGPT: Multi-Agent Framework. github.com/FoundationAgents/MetaGPT.
5. Microsoft. AutoGen: Multi-Agent Conversation Framework. github.com/microsoft/autogen.
6. CrewAI Inc. CrewAI Documentation. docs.crewai.com.